OpenPilot: Conceptual Architecture Report

EECS 4314 - Assignment 1

Group G
Harneet Arri, Gurshaan Gill, Yaohang Tang,
Oyinkansola Fejuku, Qinsong Chang

Abstract

OpenPilot is an open source driver assistance system made by comma.ai and community contributors under the MIT license. It adds lane centering, adaptive cruise control, lane change assist, and driver monitoring to cars people already own. This report looks at the conceptual architecture of OpenPilot, based on its public documentation. The system is represented as a four-step pipeline (sense, understand, plan, act), consisting of subsystems for sensor input, perception, planning, control, vehicle interface, and driver monitoring and safety. We also go over its external interfaces, meaning what information goes in and out of the system. Next, we explain how the subsystems share data through cereal, a publish subscribe messaging system, how control is coordinated, and how concurrency works since each part runs as its own process. Sequence diagrams illustrate two use cases: lane centering and driver attention alerts. We then go over the main architectural styles (publish subscribe, separate modular processes, and a safety layer between OpenPilot and the car), explain how we came up with the architecture, and look at one alternative design. Finally, we talk about what we learned about modular design, the trade offs of using messaging, and figuring out a system's architecture from its documentation.

Table of Contents

1. Introduction and Overview
2. High-Level Architecture
3. Subsystem Descriptions
4. External Interfaces
5. Data Flow, Control Flow and Concurrency
6. Use Cases
7. Sequence Diagrams
8. Architectural Styles and Design Patterns
9. Derivation Process and Alternatives
10. Developer Responsibilities
11. Lessons Learned
12. Conclusion
13. References

1. Introduction and Overview

This report looks at the conceptual architecture of OpenPilot, an open source driver assistance system made by comma.ai. A conceptual architecture goes over how the system is set up at a high level: what the main parts are, what each part does, and how they work together. The goal is to give someone who is new to OpenPilot a clear picture of how the system's parts fit together.

Some of the sources we used are the OpenPilot README, the SAFETY.md and LIMITATIONS.md documents, the comma.ai product page, and the READMEs for the cereal, panda and opendbc projects.

Our main takeaways from this assignment are that OpenPilot is made up of separate processes that talk to each other through a publish subscribe messaging system called cereal, instead of calling each other directly. The data moves through the system like a pipeline: it senses the road and the car, figures out what is going on using a neural network, plans a path and speed, and finally controls the steering, gas, and brakes. Safety of the system is enforced by the panda, a piece of hardware that is between OpenPilot and the car. Lastly, each car brand has its own module, so adding a new car is mostly a software job.

1.1 System Overview

OpenPilot is open source driver assistance software that adds lane centering and adaptive cruise control to cars people already own. Its README calls it "an operating system for robotics", and it is supported on 350+ car models.

1.2 Key Features

comma.ai lists four main features [1]:

1. Automated lane centering: keeps the car in the middle of its lane.
2. Adaptive cruise control: keeps a set speed and a safe distance from the car in front.
3. Lane change assist: changes lanes after the driver turns on the signal and nudges the wheel. It can't check the blind spot, so the driver still needs to make sure the lane is clear.
4. Driver monitoring: uses a camera facing the driver to make sure they are paying attention, instead of having them keep touching the wheel.

All these features together let the car drive for hours without intervention. However, OpenPilot is still a Level 2 system. Its safety documents say it "requires the driver to be alert and to pay attention at all times", and that "it is the driver's responsibility to be in control of the vehicle at all times" [5]. This affects the architecture, since the driver is always responsible, driver monitoring is a core part of the system.

1.3 What It Takes To Run

To use OpenPilot in a car, you need four things:

1. A comma device (the comma four or the older comma 3X)
2. The OpenPilot software
3. A supported car
4. A harness to connect the device to the car

The device mounts on the windshield. OpenPilot does not add new motors or sensors to the car. It just sends commands to the steering, gas, and brake systems that the car already has. The opendbc project explains how most cars since 2016 have electronically actuatable steering, gas, and brakes due to LKAS (Lane Keeping Assist System) and ACC (Adaptive Cruise Control). OpenPilot works by connecting to the car's CAN network.

1.4 Development and Data

OpenPilot uses the MIT license and is developed in the open on GitHub. Most of the support for new car models comes from the community, and comma.ai does the final safety and quality checks. OpenPilot also uploads driving data to commas servers, which comma uses to train better models.

2. High-Level Architecture

Figure 1 shows the conceptual architecture. Data flows left to right through the main driving pipeline: sensor input, perception, planning, control and the vehicle interface, which finally reaches the vehicle. Vehicle state returns on a feedback path, and a separate driver monitoring and safety group runs alongside the pipeline. Labels on the arrows name the main messages exchanged (VisionIPC frames, modelV2, longitudinalPlan, carControl, carState and selfdriveState).

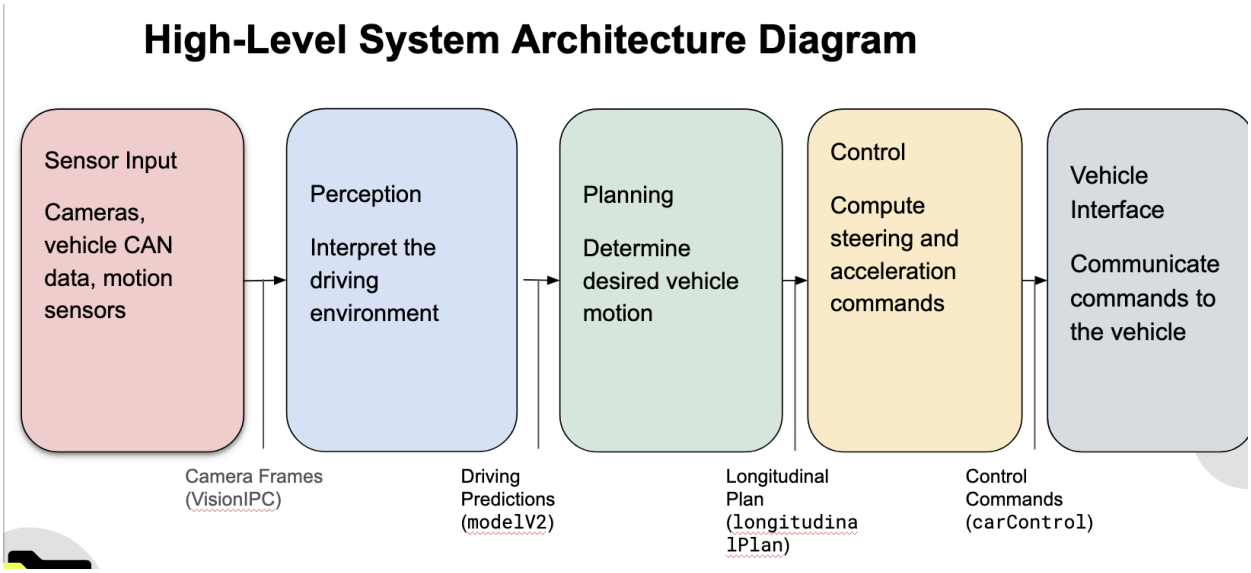


Figure 1. High-level conceptual architecture of OpenPilot. Solid arrows show the driving pipeline; dashed lines show feedback and state signals. plan = longitudinalPlan message; frames = VisionIPC camera frames.

3. Subsystem Descriptions

Subsystem	Key components	Responsibility
Sensor Input	Road and driver cameras, vehicle CAN data, motion sensors, GPS	Collects raw information about the road, the driver and the vehicle. Camera frames are delivered with VisionIPC (camerad).
Perception	camerad, modeld	Interprets the driving environment. The neural-network driving model reads camera frames and publishes driving predictions (modelV2).
Planning	Longitudinal planner	Determines the desired vehicle motion, including speed and following distance (longitudinalPlan).
Control	controlsd	Computes steering (lateral) and acceleration/braking (longitudinal) commands and publishes them as carControl.
Vehicle Interface	card, panda	Translates commands into vehicle-specific CAN messages. Panda hardware enforces safety limits on every command.
Driver Monitoring and Safety	dmonitoringmodeld, dmonitoringd, selfdrived, ui, soundd	Estimates driver attention from the cabin camera, tracks the engagement/alert state, and presents visual and audible warnings.

Cloud Link	Logging and update services	Uploads logs used to improve the model; downloads software updates.

Table 1. Subsystems and their responsibilities.

4. External Interfaces

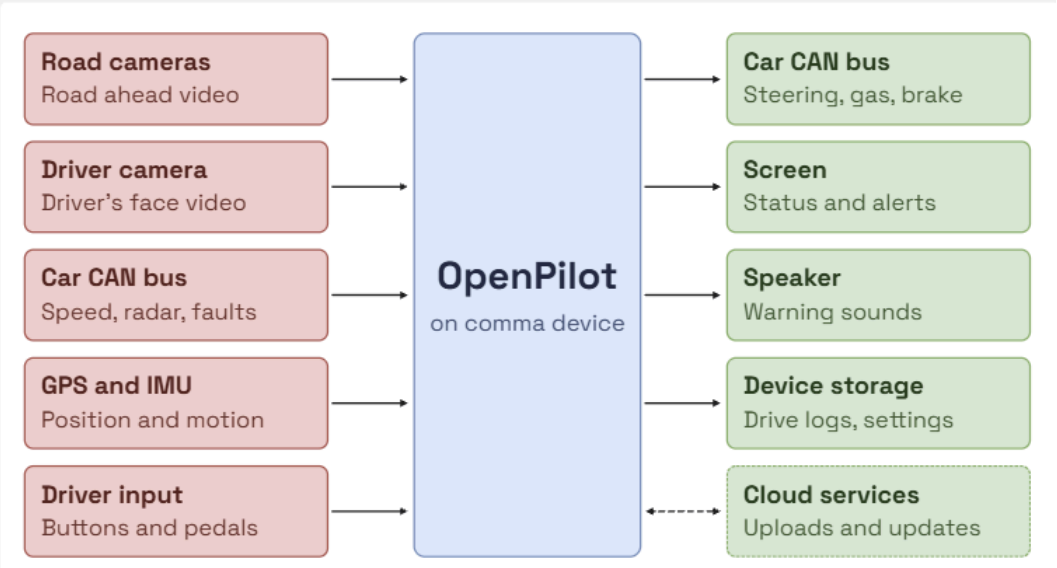


Figure 2. External interfaces of OpenPilot.

Interface	Direction	Information
Read cameras	In	Video of the road ahead
Driver camera	In	Video of the driver, used to check if they are paying attention
Car CAN bus	In	Car data like speed, steering angle, and faults
GPS and IMU	In	Position and movement of the car
Driver input	In	Pedals and steering wheel buttons
Car CAN bus	Out	Steering, gas, and brake commands
Screen and speaker	Out	Status alerts and warning sounds for the driver
Device storage	Out	Drive logs and settings

Cloud services	Both	Drive logs are uploaded and software updates are downloaded

Table 2. External interfaces of OpenPilot, showing the direction and the information exchanged at each interface.

5. Data Flow, Control Flow and Concurrency

Data flow in the openpilot system is a loop that starts at the ingestion of data from the inputs. That being in this case the camera images and vehicle state data. Once the data has been transformed into structured data by the perception and estimation sub-systems. It is transferred to the planning and control subsystems which sends the information to the final destination, that is the vehicle interface.

5.1 Data Flow

OpenPilot components exchange information through a publish/subscribe messaging system. cereal defines message schemas that specify how information is represented and shared between components. The updated vehicle-state information is used in subsequent control cycles, allowing the system to respond to changes in vehicle motion.

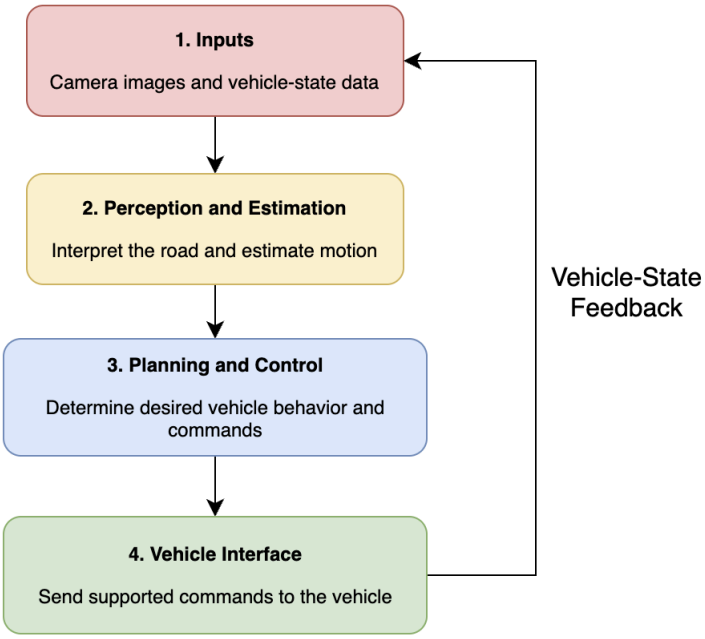


Figure 3. Showing the data flow of the system.

5.2 Control Flow

Control entails lateral control (steering) and longitudinal control (speed and acceleration/braking). The controller publishes vehicle-control information through messages. Control calculations are repeated as updated vehicle and model information becomes available. The control loop repeatedly processes updated vehicle and model information to recalculate control commands.

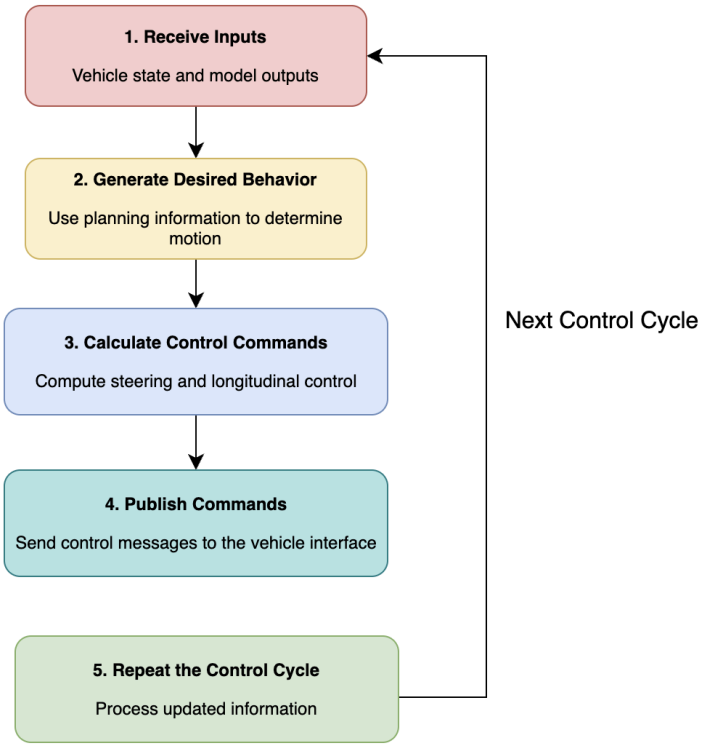


Figure 4. Showing the control cycle

5.3 Concurrency

Concurrency refers to the ability of the system to handle multiple tasks in overlapping time periods. This system can separate processes, handle different tasks and can make progress concurrently. Processes exchange information through OpenPilot's cereal publish/subscribe messaging system. Timestamps and message-validity information help components determine whether received data is usable. Delayed or unavailable data can affect downstream processing and must be handled appropriately.

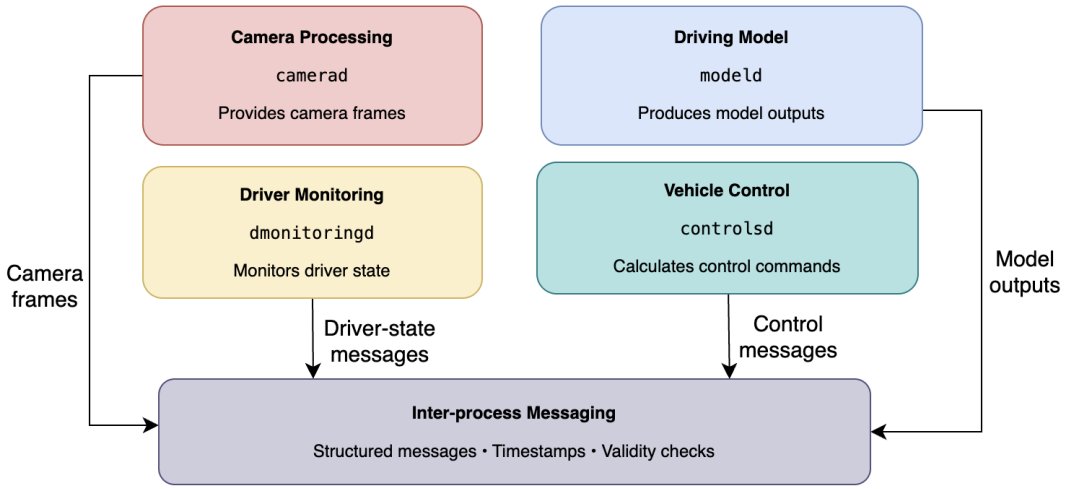


Figure 5. Showing the concurrency in the system.

6. Use Cases

6.1 Scope and Methodology

This section introduces OpenPilot from a driver's perspective. We present five use cases: engaging or disengaging assistance, maintaining lane centering, maintaining speed and following distance, assisting a lane change, and receiving driver attention alerts. We focus on two of them, lane centering (UC2) and driver attention alerts (UC5), and illustrate them with sequence diagrams in Section 7. The scope is driving assistance: according to its safety documentation, the driver has to stay attentive at all times and remains responsible for supervising the vehicle. Driving functions depend on vehicle support and configuration.[5,7]

6.2 Use Case Overview

The main actor is the driver. The driver is responsible for enabling or disabling the system, watching the road, and taking action on system warnings.

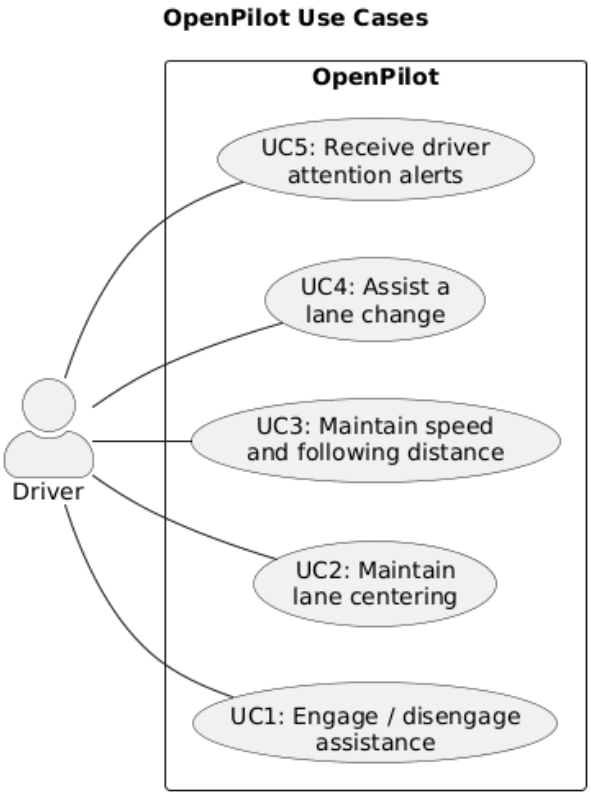


Figure 6. Relationships between the driver and different use cases inside the OpenPilot system boundary.

Use Case	Driver's Goal
UC1: Enable/Disable Assistance	Enable the system or resume manual driving.
UC2: Lane Centering	Keep the vehicle centered in the lane via steering assistance.
UC3: Speed and Following Distance Control	Maintain a set speed and a safe following distance from the vehicle ahead.
UC4: Lane Change Assistance	Receive steering assistance during a lane change.
UC5: Driver Attention Alerts	Receive a warning when not paying enough attention to road conditions

Table 3. Use cases and the driver's goal for each.

For UC1, the driver can take control at any time by pressing the brake pedal or the cancel button. For UC4, the driver nudges the steering wheel to start a lane change. OpenPilot cannot check the blind spots, so the driver must make sure the lane change is safe first. [5]

7. Sequence Diagrams

This section illustrates the two selected use cases, lane centering (UC2) and driver attention alerts (UC5), with sequence diagrams. The arrows show messages passed between components. The processes run independently and use the most recently received messages, so the order shown is conceptual.

7.1 Lane Centering (UC2)

In UC2, the driver wants the vehicle to stay centered in its lane while still monitoring the road. The preconditions are that the vehicle is supported, the device is correctly installed and calibrated, the system is enabled, and steering control is active. The function is triggered when new road images and vehicle data are received. Figure 6 shows the message flow.

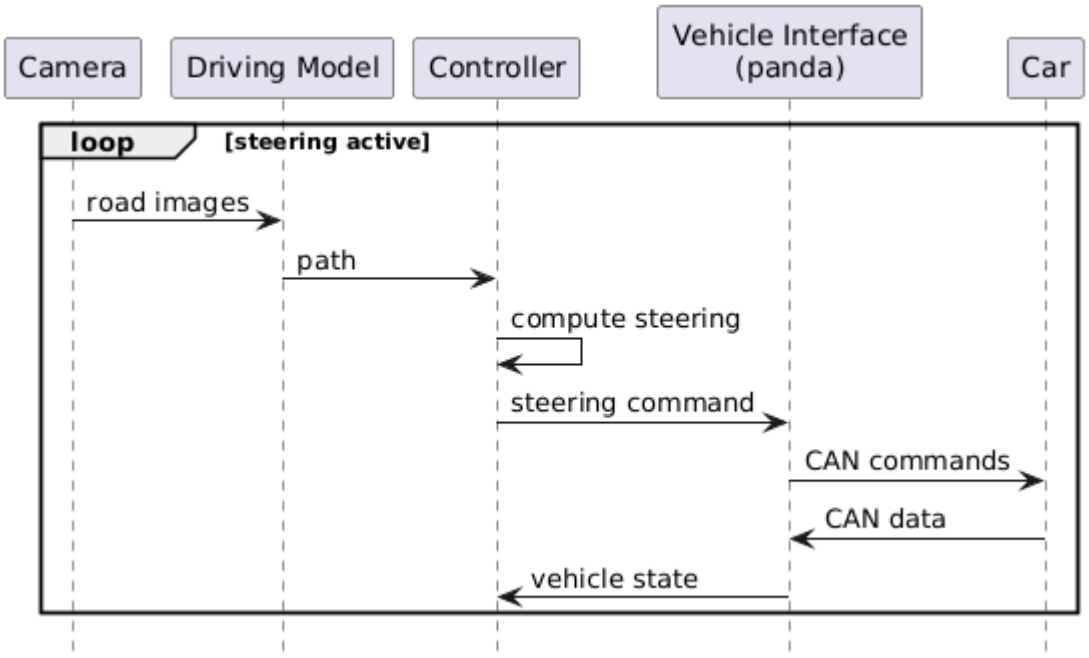


Figure 7. Sequence diagram for lane centering. The loop runs while steering is active, and each component uses the most recent input it has received.

1. The Camera sends road images to the Driving Model.
2. The Driving Model processes the images and sends a path, the trajectory the vehicle should follow, to the Controller.
3. The Controller computes the steering from this path.
4. The Controller sends a steering command to the Vehicle Interface (panda).
5. The Vehicle Interface converts the command into vehicle-specific CAN commands and sends them to the Car. Panda has built-in safety rules that can block unsafe messages.[5,8]
6. The Car sends CAN data, such as speed and steering angle, back to the Vehicle Interface. This is a continuous broadcast, not a reply to one specific command.[8]
7. The Vehicle Interface converts this CAN data into vehicle state and sends it to the Controller, which uses it in the next round of calculations.[8]

The loop runs as long as steering is active. It ends when the driver takes manual control, deactivates the system, or a system fault stops steering control.

7.2 Driver Monitoring (UC5)

In UC5, the system warns the driver when their attention is insufficient[5]. The preconditions are that the cabin camera and the driver-monitoring software are running and the system is enabled. The function is triggered whenever the DM (driver monitoring) Model produces new results. Figure 7 shows how cabin images become warnings.

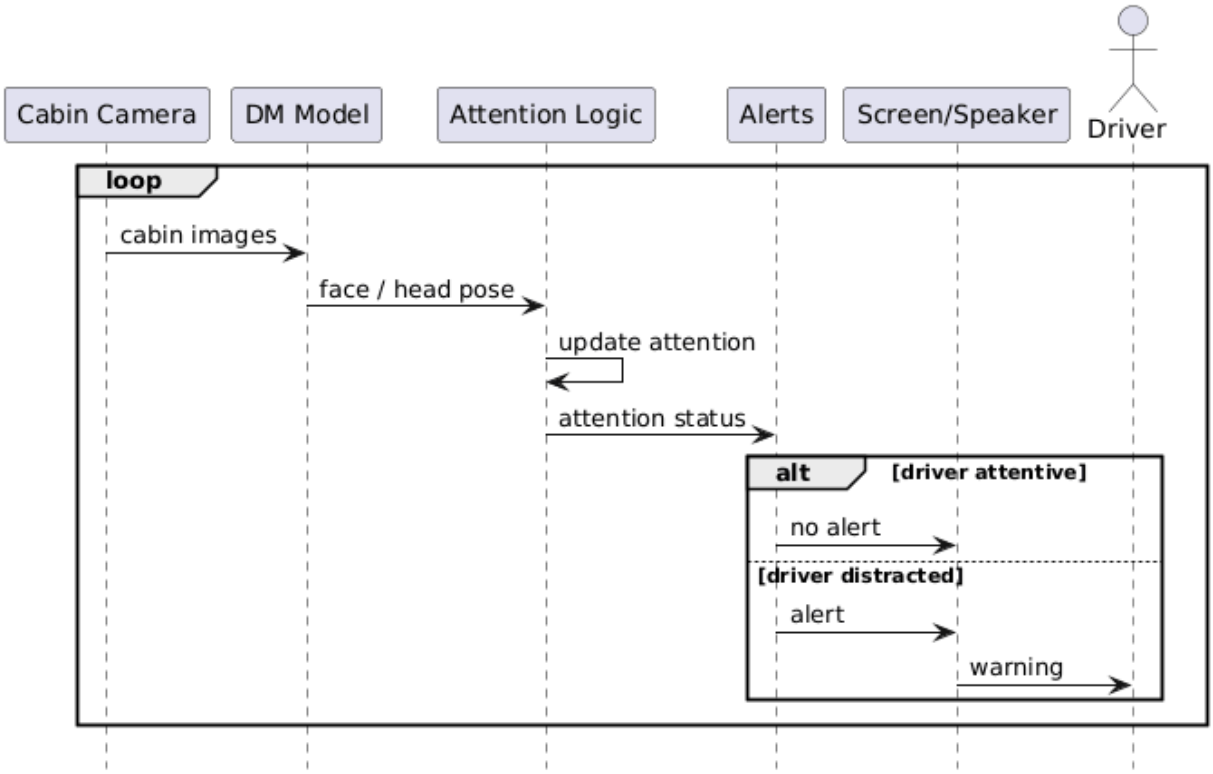


Figure 8. Sequence diagram for driver attention alerts. The loop shows that monitoring keeps running, and the alt block separates the attentive and distracted cases.

1. The Cabin Camera sends cabin images to the DM Model.
2. The DM Model analyzes the driver's face and head pose and sends the result to the Attention Logic.
3. The Attention Logic updates the driver's attention level. The level drops when the driver's gaze moves away from the road and recovers when it returns.
4. The Attention Logic sends an attention status to Alerts.
5. Alerts decides whether a warning is needed. If the driver is attentive, it sends "no alert" to the Screen/Speaker.
6. If the driver is distracted, Alerts sends an alert to the Screen/Speaker.
7. The Screen/Speaker gives the Driver a visual and audible warning.

The screen and speaker are in one lifeline because together they form the warning the driver perceives. The outer loop shows that monitoring continues after an alert, so if the driver stays distracted the warnings continue.

8. Architectural Styles and Design Patterns

Publish–Subscribe Architecture

OpenPilot uses a messaging system called cereal to support communication between its

processes. According to the official documentation, cereal uses msgq as its publish-subscribe backend and Cap'n Proto to serialize structured messages.[2]
This lets processes exchange information through messages without calling each other directly. For example, camera processing, the driving model and vehicle control are handled by different components that exchange information to deliver the driver assistance functions.
A key advantage of publish-subscribe communication is that it reduces direct dependencies between components, so each process can do its job independently. However, when message structures change, the system still has to manage message timing, data validity and compatibility carefully.

Process-Based Modular Design
OpenPilot is also built as separate modular processes, such as camerad, modeld, controlsd, card and dmonitoringd. Each has a distinct responsibility: camerad handles camera processing, modeld runs the driving model, controlsd calculates vehicle control commands, card supports communication with the vehicle, and dmonitoringd handles driver monitoring.[3]
This separation makes the architecture easier to understand and maintain and lets developers focus on specific components. However, developers must still make sure the processes communicate correctly through their shared interfaces.

Safety Layer Between Software and Vehicle
OpenPilot places a safety layer at the boundary between the software and the car. Every command to the vehicle passes through the panda hardware, which enforces safety limits. If something goes wrong in the software, the panda can still block an unsafe command. Panda's limits must stay consistent with what the control software expects or else good command could get blocked.[5,8]

9. Derivation Process and Alternatives

9.1 Derivation Process
A conceptual architecture can be derived through analysis of the system's key functions, identification of its major subsystems, and the relationship between those subsystems. The first step is reviewing OpenPilot's official documentation to understand the key functions, such as adaptive cruise control, automatic lane centering, and driver monitoring. The second step consists of defining the boundaries of the major subsystems. Based on the system's functionality, key areas include: sensor inputs, perception, planning, control, and vehicle interfaces. In the driver assistance process, each subsystem has a different function. The third step is to examine how these subsystems interact with one another. OpenPilot's cereal documentation describes its publish-subscribe messaging mechanism, and how the different processes are organized. The fourth step is to use diagrams to represent the architecture. The team's high-level architecture diagram shows the main subsystems, and sequence diagrams depict the lane centering and driver

monitoring functions. Finally, compare the conceptual diagram with the existing technical documentation. This is important as a conceptual diagram is a simplification of a complex system and may not reflect all implementation details.[1,2,3]

9.2 Alternative: Direct Module Calls

An alternative to publish-subscribe communication is direct module communication. In this design, components communicate by directly calling other components instead of exchanging information via a messaging system. Direct communication can be simpler as the relationships between components are typically easier to track. This might be suitable for small systems with limited number of components.

However, direct module communication can also lead to higher dependencies between components. When changing one component, other components may have to be modified. This makes large systems hard to maintain. In OpenPilot, publish-subscribe communication is a good fit. The system has multiple processes with different responsibilities. It allows components to exchange information while reducing direct coupling.

Publish-subscribe communication also brings challenges such as message timing, synchronization, and debugging. Therefore, neither approach is better for all situations. In this report, the direct module communication is only used as a hypothetical alternative to compare the characteristics of different architectures.

10. Developer Responsibilities

OpenPilot is split into subsystems that only talk through set interfaces, so developers have to keep each part working, and keep the parts working together. Their responsibilities are the following:

1. **Perception and models:** Developers look after the components that handle the camera inputs and produce the model outputs, and the information they pass on has to be reliable. Planning and control depend on what the model gives them, so a bad or late output affects everything after it.
2. **Planning and control:** Developers maintain the planning logic and the control calculations, and check that control decisions are based on the right model outputs and vehicle state. The controller compares what the model predicts with how the car is really moving, so its feedback loop has to keep working.
3. **Messaging and interfaces:** The processes share information only through messages, so developers keep the message definitions and communication consistent. If an interface changes, the components that use it still have to work, and changing one message can affect every component that uses it.

4. **Vehicle integration:** Every car takes commands in its own way, so developers maintain the car-specific communication and check that control commands are handled correctly by the vehicle interface. Keeping these details inside the vehicle interface is also why adding a new car is mostly a software job.
5. **Testing and debugging:** Developers find, reproduce and investigate problems, and they test the components both alone and together. Testing alone isn't enough, because many problems only show up when the processes interact, for example when a message arrives late or has different contents than a component expected. Regression testing makes sure updates don't break things that already worked. Together with clear interfaces and integration testing, it helps stop a change in one component from causing problems somewhere else in OpenPilot.

11. Lessons Learned

11.1 Modular Design

One important lesson from studying OpenPilot is that modular design helps in organizing large software systems. Separating responsibilities into different processes makes it easier to understand the overall architecture. It can also help with software maintenance since developers can focus on individual components. However, clear interfaces are still necessary to make sure that changing one component doesn't cause problems with other parts of the system.[3]

11.2 Messaging Trade-offs

Another important lesson is that publish–subscribe communication offers flexibility by reducing direct dependencies between processes. However, using a messaging system doesn't remove all communication problems. Developers need to manage message timing, data validity, and compatibility between components. This shows us that architectural decisions are often trade-offs between flexibility and complexity.[2]

11.3 Architecture Recovery

A third lesson is that recovering software architecture needs careful analysis of the available documentation. High-level architecture diagrams make complex systems easier to understand, but they do not always depict all interactions between the components. Therefore, architectural assumptions should be checked with reliable technical information. [1,2]

12. Conclusion

Overall, OpenPilot shows the power of modular processes and structured messaging for building a complex driver assistance system. The architecture separates the responsibilities of camera processing, driving models, vehicle control, driver monitoring and vehicle interfaces. The publish-subscribe communication enables the different components to share information without

the need for any direct communication between all pairs of processes. The design choices support modularity and maintainability. But real-time feedback, message timing, and safety requirements also pose some challenges that need careful.[2,3,5]

13. References

[1] comma.ai. *OpenPilot — Official Website*. https://www.comma.ai/openpilot
[2] comma.ai. *OpenPilot — cereal Messaging Documentation*.
https://github.com/commaai/openpilot/blob/master/openpilot/cereal/README.md
[3] comma.ai. *OpenPilot — Process Configuration*.
https://github.com/commaai/openpilot/blob/master/openpilot/system/manager/process_config.py
[4] comma.ai. *OpenPilot — GitHub README*. https://github.com/commaai/openpilot
[5] comma.ai. *OpenPilot — SAFETY.md*.
https://github.com/commaai/openpilot/blob/master/docs/SAFETY.md
[6] comma.ai. *OpenPilot — LIMITATIONS.md*.
https://github.com/commaai/openpilot/blob/master/docs/LIMITATIONS.md
[7] comma.ai. OpenPilot — Supported Cars. https://docs.comma.ai/CARS/
[8] comma.ai. *opendbc — README*. https://github.com/commaai/opendbc